project10_world_model_v2

Final Project: Teaching an Agent to Imagine

Learning a World Model for More Sample-Efficient RL

Course: AI / Deep Learning & Reinforcement Learning **Relevant Lectures:** Lec12 (RNN/LSTM) · Lec17 (RL Intro, Model-Based RL) · Lec18 (MDP) · Lec19 (Bellman Equation) · Lec21 (Q-learning) **Team size:** 1 student **Estimated time:** 25–35 hours **AI coding tools:** Permitted and encouraged for implementation

---

What You Will Build

Every RL algorithm you studied in Lec19–21 learns by *doing*: the agent takes an action in the real environment, observes what happens, and updates. But what if each real interaction is expensive — a robot that could break, a medical trial that costs money, a simulation that takes hours to run? A smarter agent would learn from fewer real interactions by building a mental model of how the world works, and then *practicing in its imagination*.

This project builds exactly that. You will:

1. Build a small stochastic gridworld environment
2. Train two neural networks to serve as a **world model** — one predicting the next state, one predicting the reward
3. Use the world model to generate *imagined* experience inside the **Dyna-Q** algorithm (Sutton, 1990), supplementing real environment steps with synthetic ones
4. Compare how quickly a Dyna-Q agent learns versus a plain Q-learning agent that has no world model
5. Deliberately break the world model and analyze the failure

The central question this project answers:

> *When a neural network learns to predict what will happen next in an environment, does that prediction actually help an RL agent learn faster? And what happens when the prediction is wrong?*

You are encouraged to use AI coding tools for implementation. The written analysis sections — the experimental comparison and the failure analysis — require your own reasoning about the results and cannot be generated by AI.

Background: The Dyna-Q Idea

In standard Q-learning (Lec21), each real environment step produces one training transition `(s, a, r, s')` that updates the Q-table. Learning is slow because every piece of information comes from the real world.

Dyna-Q (Lec17, Model-Based RL section) adds one step: after updating from the real transition, the agent also generates **K imagined transitions** using a learned world model, and trains on those too. Each real step now produces K+1 updates instead of 1.

```
Standard Q-learning:                    Dyna-Q:

real step → 1 Q update                  real step → 1 Q update
                                                  → K imagined updates
                                                     via world model
```

The world model is just two functions learned from data:

```
Transition model:  f(s, a) → predicted next state  s'
Reward model:      g(s, a) → predicted reward       r
```

If these predictions are accurate, the imagined transitions are nearly as good as real ones. If the predictions are inaccurate, imagined transitions may mislead the agent. The experiments in this project are designed to observe both cases.

Part 1: Build the Environment (suggested: ~4 hours)

The Gridworld

Build a stochastic 8×8 gridworld. This is the same type of environment introduced in Lec18, extended with one important addition: **noise**.

```
Grid:    8×8 cells, positions indexed as (row, col) with row,col ∈ {0,...,7}
Start:   top-left corner (0, 0) each episode
Goal:    bottom-right corner (7, 7)
Walls:   place 8–10 wall cells at fixed positions that block movement
         (design them so a path from start to goal always exists)
```

Stochastic transitions: When the agent takes action `a`, with probability **p = 0.2** the environment ignores `a` and instead moves the agent in a uniformly random direction. With probability 0.8 the intended action is executed. If movement would go into a wall or outside the grid, the agent stays in place.

Why stochasticity? A deterministic environment is trivially predictable — the world model just memorizes a lookup table. Stochasticity forces the world model to learn a genuine probability distribution over outcomes, which is the interesting and realistic case. It also makes Q-learning harder, creating more room for Dyna-Q's imagined experience to help.

```
Actions:  0=up, 1=down, 2=left, 3=right   (4 actions)

Reward:
  +1.0   on reaching the goal (7,7) — episode ends
  −0.01  every other step (small time penalty)
  Episode ends after 100 steps if goal not reached
```

State representation: The state is the integer tuple `(row, col)` — 64 possible states. This is small enough for the tabular Q-learning baseline to work perfectly, and small enough that you can visualize the entire value function as an 8×8 heatmap.

What to Implement

Create `gridworld.py` with the following class:

```python
class GridWorld:
    def __init__(self, noise_prob=0.2):
        # 8x8 grid, fixed wall positions, noise probability
        ...

    def reset(self):
        # Place agent at (0,0), sample new episode
        # Return: state as integer tuple (row, col)

    def step(self, action):
        # Apply action with noise
        # Return: (next_state, reward, done, info)
        #   info = {'true_next': (row,col), 'noisy': bool}

    def get_all_transitions(self):
        # Return the true transition matrix P[s,a,s'] and reward matrix R[s,a]
        # Used only by Value Iteration (the oracle baseline)
```

Verification checkpoint: Before writing any model or RL code, run 100 random episodes and plot the fraction of time each cell is visited as an 8×8 heatmap. The top-left region should be visited more often than the bottom-right (agent starts at top-left). Cells adjacent to walls should have slightly lower visit frequency. Include this heatmap as Figure 1 in your report.

Part 2: Build the World Model (suggested: ~5 hours)

The world model consists of two small neural networks trained on data collected from the real environment.

Step 1: Collect Training Data

Before training the world model, collect a dataset of real environment transitions using a **random policy** (take uniformly random actions):

```
dataset = []
state = env.reset()
for _ in range(10_000):
    action = random.randint(0, 3)
    next_state, reward, done, _ = env.step(action)
    dataset.append((state, action, reward, next_state))
    if done:
        state = env.reset()
    else:
        state = next_state
```

Using a random policy rather than an optimal one is intentional: you want the dataset to cover the full state space, not just the states the optimal policy visits. Split the dataset 80/20 into training and validation sets.

Why a random policy? If you collected data with a trained agent, it would only visit states near the optimal path. The world model would then never see most of the state space, and imagined transitions from unexplored states would be completely wrong.

Step 2: Represent States and Actions as Vectors

The state `(row, col)` must be converted to a vector before feeding it to a neural network. Use a simple one-hot encoding over all 64 grid positions:

```
def state_to_vector(state):
    # state: (row, col)
    idx = state[0] * 8 + state[1]     # convert to integer index 0..63
    vec = np.zeros(64, dtype=np.float32)
    vec[idx] = 1.0
    return vec

def action_to_vector(action):
    vec = np.zeros(4, dtype=np.float32)
    vec[action] = 1.0
    return vec
```

The input to both world model networks is the **concatenation** of the state vector (64 values) and action vector (4 values) — a total of 68 input values.

Step 3: Train the Transition Network

The transition network predicts the next state given the current state and action.

```
Architecture:
  Input:  68 values  (state one-hot + action one-hot)
  Layer1: Linear(68, 256) → ReLU
  Layer2: Linear(256, 256) → ReLU
  Output: Linear(256, 64)    ← logits over 64 possible next states
  Loss:   CrossEntropyLoss(predicted_logits, true_next_state_index)
```

This is a **classification** problem: given (s, a), predict which of the 64 grid cells comes next. CrossEntropy loss is appropriate here — the same loss used for image classification in Lec11.

Train for 20 epochs on the training set. After each epoch, compute validation accuracy (fraction of transitions where the argmax of the output matches the true next state). A well-trained model on this environment should reach 75–85% validation accuracy — not 100%, because the 20% noise means some transitions are genuinely unpredictable.

Step 4: Train the Reward Network

The reward network predicts the scalar reward given the current state and action.

```
Architecture:
  Input:  68 values  (state one-hot + action one-hot)
  Layer1: Linear(68, 128) → ReLU
  Layer2: Linear(128, 64) → ReLU
  Output: Linear(64, 1)     ← scalar reward prediction
  Loss:   MSELoss(predicted_reward, true_reward)
```

Train for 20 epochs. Reward prediction should be easier than transition prediction — most transitions give −0.01 (highly predictable), and only transitions into (7,7) give +1.0.

Step 5: Verify the World Model

Before using the world model for planning, verify it works:

- Pick 5 specific (state, action) pairs. For each one, query the transition network and show the top-3 predicted next states (highest softmax probabilities). Do the predictions make physical sense? (e.g., action=right from cell (3,4) should most likely predict (3,5), with some probability on (3,4) for wall bounce or noise).
- Plot validation accuracy vs. epoch for the transition network. Include this as Figure 2 in your report.

Part 3: Implement Dyna-Q (suggested: ~5 hours)

The Algorithm

Dyna-Q augments standard Q-learning with imagined experience. The Q-table has shape (64, 4) — one value per (state, action) pair.

```
        def dyna_q(env, transition_model, reward_model,
                   K=10, alpha=0.1, gamma=0.95,
                   n_steps=50_000, epsilon_start=1.0, epsilon_end=0.05):

            Q = np.zeros((64, 4))            # Q-table
            replay_buffer = []               # stores real transitions seen so far

            state = env.reset()
            epsilon = epsilon_start

            for step in range(n_steps):
                # --- Real environment step ---
                epsilon = max(epsilon_end, epsilon_start - step / n_steps * (epsilon_start -
epsilon_end))

                if random.random() < epsilon:
                    action = random.randint(0, 3)
                else:
                    action = np.argmax(Q[state_to_index(state)])

                next_state, reward, done, _ = env.step(action)

                # Real Q-update (standard Q-learning from Lec21)
                s  = state_to_index(state)
                s_ = state_to_index(next_state)
                Q[s, action] += alpha * (reward + gamma * np.max(Q[s_]) - Q[s, action])

                replay_buffer.append((state, action))    # remember what we've seen
                if done:
                    state = env.reset()
                else:
                    state = next_state

                # --- K imagined steps using the world model ---
                for _ in range(K):
                    # Sample a previously-seen (state, action) pair
                    img_s, img_a = random.choice(replay_buffer)

                    # Ask the world model what would happen
                    img_s_vec = concat(state_to_vector(img_s), action_to_vector(img_a))
                    img_s_next_logits = transition_model(img_s_vec)
                    img_s_next = argmax(softmax(img_s_next_logits))   # sample next state

                    img_r = reward_model(img_s_vec).item()

                    # Imagined Q-update (same formula as real update)
                    s_i  = state_to_index(img_s)
                    s_i_ = img_s_next
                    Q[s_i, img_a] += alpha * (img_r + gamma * np.max(Q[s_i_]) - Q[s_i,
img_a])

            return Q
```

One key design decision: When sampling imagined transitions, we only sample from `(state, action)` pairs the agent has actually visited. This prevents the world model from being queried on states it has never seen — where its predictions would be unreliable. This is important: explain it in your report.

The Baseline: Plain Q-learning

Implement standard Q-learning as a special case of Dyna-Q with K=0:

```
        def q_learning(env, alpha=0.1, gamma=0.95, n_steps=50_000, ...):
            return dyna_q(env, transition_model=None, reward_model=None, K=0, ...)
```

No world model, no imagined steps — pure real-environment learning.

The Ceiling: Value Iteration

Implement Value Iteration (Lec19) using the *true* transition probabilities from `env.get_all_transitions()`. This gives the optimal Q-values without any trial-and-error learning:

```
        def value_iteration(P, R, gamma=0.95, threshold=1e-6):
            # P[s, a, s'] = transition probability
            # R[s, a]     = expected reward
            V = np.zeros(64)
            while True:
                V_new = np.max(R + gamma * np.sum(P * V[np.newaxis, np.newaxis, :], axis=2),
axis=1)
                if np.max(np.abs(V_new - V)) < threshold:
                    break
                V = V_new
            return V
```

Value Iteration has access to the true environment dynamics — no learning required. It represents the **performance ceiling**: the best policy that any amount of real-world experience could possibly produce. Compare your learned agents against it.

Part 4: Run the Main Experiment (suggested: ~5 hours active, several hours compute)

What to Run

Train three agents on the same gridworld environment with the same hyperparameters:

Agent	Description
Value Iteration	Oracle — uses true P(s'|s,a), no learning steps needed
Q-learning (K=0)	No world model, only real experience
Dyna-Q (K=10)	10 imagined steps per real step using your trained world model

For Q-learning and Dyna-Q, use **3 random seeds** each. Record every 500 steps:

- Average episode reward over the last 20 episodes
- Success rate: fraction of last 20 episodes where the agent reached the goal

Important: The world model is trained *once* on the 10,000-transition random-policy dataset before any RL training begins. The world model weights are then frozen — they do not update during Q-learning or Dyna-Q. This isolates the effect of the world model from any online adaptation.

What to Plot

Figure 3 — Learning curves (required): Plot **success rate vs. real environment steps** for all three agents. For Q-learning and Dyna-Q, show the mean across 3 seeds as a solid line with ±1 std shading. Draw a horizontal dashed line at the Value Iteration ceiling (its success rate evaluated over 200 test episodes).

Figure 4 — Value function comparison (required): Show three 8×8 heatmaps side by side:

- Value Iteration V*(s) — the ground truth optimal values
- Q-learning V(s) = max_a Q(s,a) at convergence (200K steps)
- Dyna-Q V(s) = max_a Q(s,a) at convergence (200K steps)

Use the same color scale for all three. Wall cells should be shown in gray. The goal cell (7,7) should have the highest value in all three heatmaps.

What to Write: Experiment Analysis (500–700 words)

Answer all of the following questions based on your actual results. Be specific — include numbers from your figures.

1. **At 50,000 real environment steps, what is the success rate of Q-learning vs. Dyna-Q?** How large is the gap? Express it as an absolute difference (e.g., "Dyna-Q achieved 62% vs. Q-learning's 41% at 50K steps"). Did Dyna-Q converge faster, reach a higher final success rate, or both?

2. **How does the Dyna-Q value function (Figure 4) compare to Value Iteration?** Find two cells where they differ most. Is the difference larger near the start cell, near the goal, or near the walls? Why might those regions be hardest for Dyna-Q to learn correctly?
3. **What does K=10 mean in practice?** At 50,000 real environment steps, Dyna-Q has performed 50,000 real Q-updates plus 500,000 imagined Q-updates. That is equivalent (in number of Q-updates) to Q-learning running for 550,000 steps. Does Dyna-Q's actual performance at 50K real steps match Q-learning at 550K real steps? If not, why not? (Think about what's different between a real transition and an imagined one.)
4. **The world model has 75–85% transition accuracy — not 100%.** For every 10 imagined transitions, roughly 2 are wrong. Does this level of inaccuracy seem to hurt Dyna-Q in your results? Look at the variance (the std shading in Figure 3) — is Dyna-Q's variance larger or smaller than Q-learning's? What does this tell you about how model error affects learning stability?
5. **Connect to the Bellman equation.** Write out the Q-learning update rule from Lec21. Now write out what the imagined update in Dyna-Q looks like when the transition model predicts the wrong next state `ŝ'` instead of the true `s'`. What value does the agent bootstrap from? Is this a problem if `ŝ'` is always just one cell away from the true `s'`?

Part 5: Forced Failure Analysis (suggested: ~6 hours)

You must complete this section. It is worth 25% of the grade and requires deliberately breaking your system, observing the failure, and explaining it mechanistically using the theory from the course.

The Failure Experiment

The setup: Train a **bad world model** by using only a tiny dataset — instead of 10,000 transitions, use only **200 transitions** from the random policy. Everything else stays identical: same architecture, same training procedure, same number of epochs, same Dyna-Q hyperparameters (K=10).

Call this agent **Dyna-Q (bad model)**. Run it for 200,000 real environment steps with 3 seeds, exactly like your main experiment.

Your prediction (write this before running): Based on what you know about how Dyna-Q works, do you expect the bad model to perform better than, worse than, or about the same as plain Q-learning (K=0)? Write one paragraph stating your prediction and the reasoning behind it. Submit this before you run the experiment.

What to Plot

Add **Dyna-Q (bad model)** as a fourth line to Figure 3 (the learning curves from Part 4), using a dashed line style to visually distinguish it.

What to Write (500–700 words)

Answer all of the following:

1. Was your prediction correct? State your original prediction and whether the results matched it. If you were wrong, describe what surprised you. (There is no grade penalty for a wrong prediction — only for not making one or for not analyzing the discrepancy honestly.)

2. Explain the failure mechanistically. A bad world model trained on 200 transitions has seen only a small fraction of the 64 × 4 = 256 possible (state, action) pairs. For pairs it has never seen, its predictions are essentially random — it outputs whatever the network's weights happen to produce for that input.

When Dyna-Q samples an imagined transition from one of these unseen pairs, it generates a fake Q-update. Describe in your own words what happens to the Q-table entry for that pair over many such fake updates. Does it get pushed toward a high value? A low value? Random values? Connect this to the Q-learning update formula: `Q(s,a) ← Q(s,a) + α * (r̂ + γ * max Q(ŝ') − Q(s,a))`. If `r̂` and `ŝ'` are both wrong, what values is `Q(s,a)` being pulled toward?

3. Where in the gridworld does the bad model fail most? Compare the 8×8 value function heatmaps: Value Iteration V*, good Dyna-Q, and bad Dyna-Q. Find the 3 cells where bad Dyna-Q's value differs most from V*. Are these cells in the part of the grid that was well-covered by the 200-transition dataset (near the start cell) or poorly covered (far from the start)? Why does coverage matter?

4. Quantify the dataset size effect. Run one additional experiment: train the world model on **2,000 transitions** (ten times more than the bad model, five times less than the good model) and evaluate Dyna-Q with this intermediate model. Plot its success rate curve alongside the other conditions. Does performance improve smoothly as you add more data, or is there a threshold effect? Report the success rate at 50,000 real steps for all four dataset sizes: 200, 2,000, and 10,000 transitions, plus the K=0 baseline. Present these as a bar chart (Figure 5).

5. Propose a practical fix. Suppose you are in a real application where you can only collect 200 initial transitions — data collection is genuinely expensive. What could you do to make the world model more reliable without collecting more data upfront? Propose one concrete technique. Options to consider:

- **Uncertainty-aware planning:** Only use the world model for imagined transitions where the model is confident (high softmax probability on the predicted next state).

Discard imagined transitions where the model is uncertain. Explain how this would change the Dyna-Q algorithm.

- **Conservative imagination:** Weight imagined Q-updates by the model's prediction confidence — confident imagined transitions count as much as real transitions; uncertain ones count for almost nothing. Write the modified update rule.
- **Online model improvement:** Instead of freezing the world model, continue training it on new transitions collected during RL training. Explain the risk of this approach (hint: the RL policy changes over time, so the distribution of visited states changes — does the world model stay calibrated?).

Pick one option, explain the mechanism clearly, and argue why it would specifically help in the 200-transition setting.

Part 6: Figures and Report (suggested: ~5 hours)

Required Figures

Figure	Content	Location
Figure 1	8×8 state visitation heatmap from random policy	Part 1 verification
Figure 2	Transition model validation accuracy vs. training epoch	Part 2 verification
Figure 3	Learning curves: success rate vs. real steps (all 4 conditions)	Parts 4 & 5
Figure 4	3-panel value function heatmaps: VI, Q-learning, Dyna-Q	Part 4
Figure 5	Bar chart: success rate at 50K steps vs. dataset size	Part 5

All figures must have axis labels, a title, and a 1–2 sentence caption.

Report Structure

Section 1 — Introduction (200–300 words) What problem does Dyna-Q solve? Why is sample efficiency important in RL? What were your key findings?

Section 2 — Environment (250–350 words) Describe the gridworld: state space, action space, reward function, stochastic transitions. Write the MDP tuple (S, A, R, γ) explicitly. Include Figure 1. Explain why stochasticity makes the world model problem harder and more interesting.

Section 3 — World Model (350–450 words) Describe the transition and reward networks: architecture, inputs, outputs, loss functions. Explain why CrossEntropy loss is appropriate for transition prediction. Include Figure 2. Discuss why validation accuracy is ~80% rather than 100% — is this a failure of the model or a fundamental property of the environment?

Section 4 — Dyna-Q Algorithm (300–400 words) Describe Dyna-Q in plain language. Include the pseudocode or the key loop from Part 3. Explain the role of K and why imagined transitions are only sampled from previously-visited (state, action) pairs.

Section 5 — Main Experiment Results (500–700 words) Answer the 5 analysis questions. Include Figures 3 and 4.

Section 6 — Failure Analysis (500–700 words) Include your pre-experiment prediction, the results, and all 5 answers. Include Figures 4 (bad model panel) and 5.

Section 7 — Reflection (200–300 words) What was the hardest part? What would you do differently? What is one thing you now understand about model-based RL that you did not before?

Section 8 — AI Tool Usage Log (100–200 words) Which parts did AI tools generate? Which did you correct? What was wrong with any generated code?

Total report length: approximately 2,300–3,400 words, plus figures.

Grading Rubric

Component	Points	Description
Environment implementation	10	Correct grid, walls, stochastic transitions, reward
World model training	12	Both networks; correct loss functions; validation plot
Dyna-Q implementation	13	Correct Q-updates; replay buffer sampling; K imagined steps
Figure 1 (visitation map)	4	Correctly generated and interpreted
Figure 2 (model accuracy)	4	Shows training curve; correct accuracy numbers
Figure 3 (learning curves)	8	3 seeds; mean ± std; all 4 conditions
Figure 4 (value heatmaps)	6	Correct color scale; all 3 panels
Main experiment analysis	15	5 questions answered with specific numbers and reasoning
Pre-experiment prediction	5	Written before running; honest
Failure analysis — empirical	8	Experiment run; numbers reported; Figure 5 present
Failure analysis — mechanistic	10	Explains *why* using Lec18/19/21 concepts
Failure analysis — proposed fix	5	Technically coherent; addresses the mechanism
Report writing quality	5	Follows structure; clear captions; correct lecture citations
AI usage log	5	Honest and specific
Total	**110**	*(scaled to 100 in final grade)*

Practical Tips

On training time: The world model trains quickly — 20 epochs on 8,000 transitions takes under 2 minutes on CPU. The RL training (200,000 steps × 3 seeds × 4 conditions = 12 runs) takes roughly:

- ~5 minutes per run on CPU (tabular Q-learning is fast)
- Total: ~1 hour of compute for all runs

Unlike Project 1, there is no GPU bottleneck here — everything is tabular or small MLP. Run all experiments in a single afternoon.

On implementing Value Iteration: The `get_all_transitions()` method returns the true P[s,a,s'] array. Value Iteration converges in under 1 second for a 64-state MDP. Use its converged V* both as a baseline in Figure 3 (evaluate the greedy policy derived from V* on 200 test episodes) and as the ground-truth color scale in Figure 4.

On the failure analysis prediction: Write the prediction paragraph in a separate document or a clearly marked comment at the top of your failure experiment script *before running anything*. Screenshot it or save it with a timestamp. This is the most non-delegatable part of the project — an AI tool cannot make this prediction for you because the results do not exist yet.

On debugging a non-improving Dyna-Q: If Dyna-Q performs the same as Q-learning, the most common causes are: (1) the world model was not trained (check that loss decreased during training), (2) imagined transitions are being sampled from states with index 0 only (check the replay buffer sampling), or (3) K is accidentally set to 0 in the function call.

Connection to Course Lectures

Project element	Lecture
MDP formulation (S, A, R, P, γ)	Lec18: MDP definition, Markov property
Value Iteration as oracle	Lec19: Bellman optimality, value iteration algorithm
Q-learning update rule	Lec21: TD learning of action values, Q-learning
ε-greedy exploration	Lec17: Exploration-exploitation tradeoff
Model of environment (transition + reward)	Lec17: Model-based RL, model of environment slide
Dyna-Q concept	Lec17: Model-based RL, training efficiency section
Neural network as function approximator	Lec08: Neural networks, Lec11: why CNNs generalize
CrossEntropy loss for classification	Lec08/11: Softmax output, classification losses
Why model error hurts: bootstrapping from wrong values	Lec19: Bellman equation, Lec21: TD error
One-hot encoding, vector representations	Lec08: Neural network inputs

SJTU AI / Deep Learning & Reinforcement Learning Course Submission: Code (zipped), Report (PDF), AI usage log (included in report)